Data Containerization

Docker Architecture: https://docs.docker.com/get-started/overview/ (https://docs.docker.com/get-started/overview/)

Go Programming: https://go.dev/ (https://go.dev/)

Docker Containerization

Building Blocks of the Docker Stack:

- **Docker CLI.**
- **Docker Engine.**
- **Docker Compose:**
  - Compose has commands for managing the whole lifecycle of your application:
    - Start, stop, and rebuild services
    - View the status of running services
    - Stream the log output of running services
    - Run a one-off command on a service

Documentations: https://docs.docker.com/desktop/) (https://docs.docker.com/desktop/)) :

- **Docker Engine (https://docs.docker.com/engine/)**
- Docker CLI client
- **Docker Buildx (https://docs.docker.com/build/)**
- **Extensions (https://docs.docker.com/desktop/extensions/)**
- **Docker Compose (https://docs.docker.com/compose/)**
- **Docker Content Trust (https://docs.docker.com/engine/security/trust/)**
- **Kubernetes (https://github.com/kubernetes/kubernetes/)**
- **Credential Helper (https://github.com/docker/docker-credential-helpers/)**

Docker Capabilities

1. **File Composition/Formatting**
2. **Command-Line Interfacing**
3. **Application Programming Interfaces (APIs)**
   1. **The Docker Engine API**
      Docker Engine is an open source containerization technology for building and containerizing your applications. Docker Engine acts as a client-server application with:
      - A server with a long-running daemon process
      - APIs which specify interfaces that programs can use to talk to and instruct the Docker

daemon.
- A command line interface (CLI) client

2. **The Docker Registry HTTP API => Interactions with Instances of the Docker Registry**
   Docker Registry is a stateless, highly scalable server-side application that stores and lets you distribute Docker images.
   - tightly control where your images are being stored
   - fully own your images distribution pipeline
   - integrate image storage and distribution tightly into your in-house development workflow
3. **The Docker Hub API => Docker Hub Interactions**
   Docker Hub (https://hub.docker.com/) is a service provided by Docker for finding and sharing container images with your team. It's the world's largest repository of container images with an array of content sources including container community developers, open source projects and independent software vendors (ISV) building and distributing their code in containers.
4. **The DVP Data API => Verified Publisher Analytics Interactions**

4. **Drivers and Specifications**
   1. **Image Specification**
   2. **Registry Token Authentication**
   3. **Registry Storage Drivers**

Docker System Operations:

- Show the version of the Docker stack: **docker version**
- Show information about the stack including the server: **docker info**
- Login to a Docker registry: **docker login**
- Log out: **docker logout**
- Show low-level information about an object: **docker inspect** the_object
- Remove unused images, containers, and networks: **docker system prune**

Docker Networks Operations:

- View available networks: d**ocker network ls**
- Remove a network: **docker network rm** network_name
- Show information about a network: **docker network inspect** network_name
- Connect a container to a network: **docker network connect** network_name container_name
- Disconnect a container from a network: **docker network disconnect** networ_name container_name

Docker Plugin Operations:

- Enable a Docker plugin: **docker plugin enable [plugin]**
- Disable a Docker plugin: **docker plugin disable [plugin]**
- Create a plugin from config.json and rootfs: **docker plugin create [plugin] [path-to-data]**
- View details about a file system: **docker plugin inspect [plugin]**

- Remove a plugin: **docker plugin rm [plugin]**

Components of the Docker Desktop (A GUI for managing images, containers and applications).

Three Views

The **Images** view

displays a list of your Docker images

- allows you to run an image as a container,
- pull the latest version of an image from Docker Hub, and
- inspect images.
- it displays a summary of the vulnerability scanning report using Snyk. In addition, the **Images** view contains clean-up options to remove unwanted images from the disk to reclaim space. If you are logged in, you can also see the images you and your organization have shared on Docker Hub. For more information, see **Explore your images (https://docs.docker.com/desktop/use-desktop/images/)** .
- Image Operations:

Image Operations (docker ==>)

- Build Image: **docker build** docker_file_path
- Build image at current path: **docker build .**
- Build an image from a file: **docker build -f** file_name
- Pull an image from a repository: **docker pull** image_name
- Push an image to a registry: **docker push** image_name
- Create an image from a tarball: **docker import** URL/File_name
- Create an image from a container: **docker commit** container_name image_name
- Tag an image: **docker tag** image_name image_name:tag
- List all locally stored top-level images: d**ocker images**
- Show the history of an image: **docker history** image_name
- Remove an image: **docker rmi** image_name
- Load an image from a docker tar archive file: **docker load --image** tar_file_name
- Save an image to a docker tar archive file: **docker save** image_name **>** tar_file_name
- **Search docker hub for images: docker search** query
- Remove all unused images/images that do not have any containers associated with them : **docker image prune -a**
- Sort Images
  - In use
  - Unused
  - Dangling

The **Containers** view

provides a runtime view of all your containers and applications.

- It allows you to interact with containers and applications, and manage the lifecycle of your applications directly from your machine.
- This view also provides an intuitive interface to perform common actions to inspect, interact with, and manage your Docker objects including containers and Docker Compose-based applications. For more information, see **Explore running containers and applications (https://docs.docker.com/desktop/use-desktop/container/)** .

- ## Container Operations
  - List all running containers: **docker ps -a**
  - List the running and stopped all containers: **docker ps -s**
  - Create a container: **docker create image_name**
  - create a container with a pseudo tty: **docker create -it** image_name
  - Rename **docker rename** container_name new_name
  - List port numbers
  - Force remove a running container: **docker rm** -f container_nme
  - Remove a stopped container: **container rm** container_name
  - Pause/Resume
  - Stop/Start
  - Delete
  - Open the application in VS code
  - Open the port exposed by the container in a browser
  - Copy docker run. This allows you to easily share container run details or modify certain parameters

The **Volumes** view

- displays a list of volumes and allows you to easily create and delete volumes and see which ones are being used. For more information, see **Explore volumes (https://docs.docker.com/desktop/use-desktop/volumes/)** .

- ## Volume Operations
  - Search
  - Sort
    - Name
    - Dated Created
    - Size
    - Remove

```
C:\Users\khali>docker search helloworld
NAME                                        DESCRIPTION                                     STARS     OFFICIAL   AUTOMATED
supermanito/helloworld                      学习资料                                          270
karthequian/helloworld                      A simple helloworld nginx container to get y…   18                   [OK]
strm/helloworld-http                        A hello world container for testing http bal…   6                    [OK]
buoyantio/helloworld                                                                        4
helloworldcontainers/java-spring-boot-hello-world                                           4
ibmcom/helloworld                           A sample used by IBM Cloud Code Engine          2
wouterm/helloworld                           A simple Docker image with an Nginx server …   1                    [OK]
hakanssonlasse/helloworld                                                                   1
testcontainers/helloworld                   A Docker image to support Testcontainers' se…   1
helloworld2121/ashish                       hi                                              1
helloworld9x/kubia                                                                          0
hivetechnologies/helloworldarm              Helloworld Webserver written in GoLang for t…   0
cstxmohammad/helloworld                     to run the image use following command:  doc…   0
apache/causeway-app-helloworld              Apache Causeway App Helloworld                  0
agrawals18/helloworld                                                                       0
neyahpeterson/helloworldui                                                                  0
nimak/helloworld                                                                            0
helloworld102444/studentserver                                                              0
containerinstance/helloworld                Demo image for SSL connection inside Node.js    0
isaactl/helloworld                                                                          0
moseers/helloworld                                                                          0
neyahpeterson/helloworldapi                                                                 0
mperel/helloworld                                                                           0
simongdavies/helloworld-aci-cnab                                                            0
helloworld2121/walmart21                                                                    0
```

Docker Image/Service Composition Example:

Step 1: Define the application dependencies

a. app.py

b. Requirements.txt

Step 2: Create a Dockerfile

app.py | docker-compose.yml | Dockerfile

```
# syntax=docker/dockerfile:1
FROM python:3.7-alpine
WORKDIR /code
ENV FLASK_APP=app.py
ENV FLASK_RUN_HOST=0.0.0.0
RUN apk add --no-cache gcc musl-dev linux-headers
COPY requirements.txt requirements.txt
RUN pip install -r requirements.txt
EXPOSE 5000
COPY . .
CMD ["flask", "run"]
```

- Build an image starting with the Python 3.7 image.
- Set the working directory to `/code`.
- Set environment variables used by the `flask` command.
- Install gcc and other dependencies

- Copy `requirements.txt` and install the Python dependencies.
- Add metadata to the image to describe that the container is listening on port 5000
- Copy the current directory `.` in the project to the workdir `.` in the image.
- Set the default command for the container to `flask run`.

Step 3: Define services in a Compose file

app.py docker-compose.yml Dockerfile

```
version: "3.9"
services:
  web:
    build: .
    ports:
      - "20000:5000"
  redis:
    image: "redis:alpine"
```

This Compose file defines two services: `web` and `redis`.

The `web` service uses an image that's built from the `Dockerfile` in the current directory. It then binds the container and the host machine to the exposed port, `8000`. This example service uses the default port for the Flask web server, `5000`.

The `redis` service uses a public **Redis** (https://registry.hub.docker.com/_/redis/) image pulled from the Docker Hub registry.

Step 4: Build and run your app with Compose

`docker compose up`

Step 5 Push the image to the Docker Hub

`a. Create a web version of the image and tag it`

`docker tag` `image_id` `your_hub_name/hub_repository_name:tag`

Example: docker tag a5e0a24a719 abuosbakh/cs490:ver1.0

`b.List images`

`docker images`

`c. Push image to the Docker Hub`

`docker push your_hub_name/hub_repository_name:tag`

Example: docker push abuosbakh/cs490:ver1.0

d. pull the following image to your Docker Desktop:

docker pull abuosbakh/cs490:ver1.0

Docker System Operations:

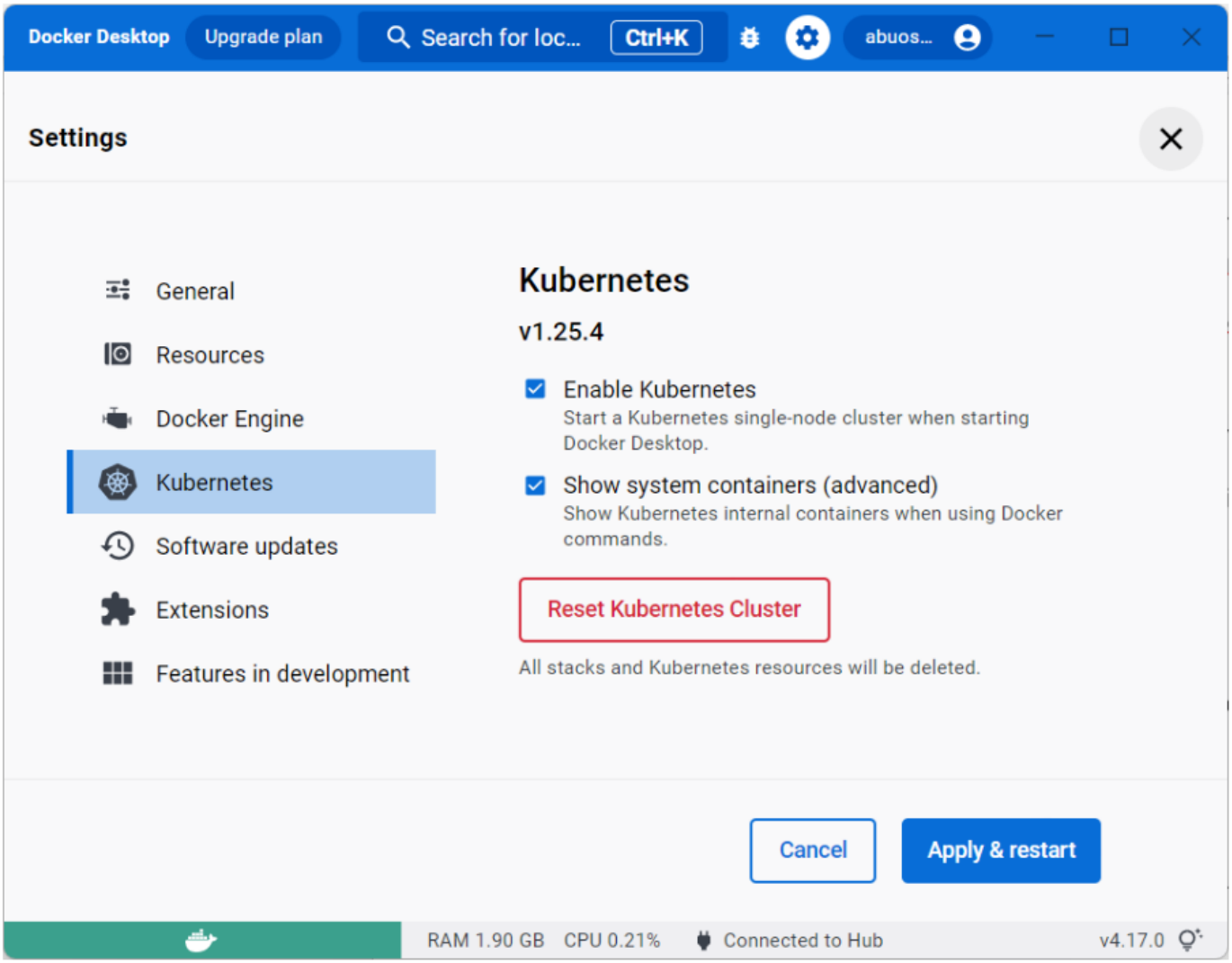
- Show the version of the Docker stack: **docker version**
- Show information about the stack including the server: **docker info**
- Login to a Docker registry: **docker login**
- Log out: **docker logout**
- Show low-level information about an object: **docker inspect** the_object
- Remove unused images, containers, and networks: **docker system prune**

Docker Networks Operations:

- View available networks: d**ocker network ls**
- Remove a network: **docker network rm** network_name
- Show information about a network: **docker network inspect** network_name
- Connect a container to a network: **docker network connect** network_name container_name
- Disconnect a container from a network: **docker network disconnect** networ_name container_name

Docker Plugin Operations:

- Enable a Docker plugin: **docker plugin enable [plugin]**
- Disable a Docker plugin: **docker plugin disable [plugin]**
- Create a plugin from config.json and rootfs: **docker plugin create [plugin] [path-to-data]**
- View details about a file system: **docker plugin inspect [plugin]**
- Remove a plugin: **docker plugin rm [plugin]**



Kubernetes, also known as K8s, is an open-source system for automating deployment, scaling, and management of containerized applications.

It groups containers that make up an application into logical units for easy management and discovery. Kubernetes builds upon 15 years of experience of running production workloads at Google, combined with best-of-breed ideas and practices from the community.

Automated rollouts and rollbacks

Kubernetes progressively rolls out changes to your application or its configuration, while monitoring application health to ensure it doesn't kill all your instances at the same time. If something goes wrong, Kubernetes will rollback the change for you. Take advantage of a growing ecosystem of deployment solutions.

Service discovery and load balancing

No need to modify your application to use an unfamiliar service discovery mechanism. Kubernetes gives Pods their own IP addresses and a single DNS name for a set of Pods, and can load-balance across them.

Storage orchestration

Automatically mount the storage system of your choice, whether from local storage, a public cloud provider such as AWS or GCP, or a network storage system such as NFS, iSCSI, Ceph, Cinder.

Self-healing

Restarts containers that fail, replaces and reschedules containers when nodes die, kills containers that don't respond to your user-defined health check, and doesn't advertise them to clients until they are ready to serve.

Secret and configuration management

Deploy and update secrets and application configuration without rebuilding your image and without exposing secrets in your stack configuration.

Automatic bin packing

Automatically places containers based on their resource requirements and other constraints, while not sacrificing availability. Mix critical and best-effort workloads in order to drive up utilization and save even more resources.